



Project Report

Project Title: Scheduler Manipulator — Cheat the OS

Subtitle: Real-Time Linux Kernel Scheduler Manipulation Tool

Course: Operating Systems

Submitted To: Sir Mahaz Khan

Submitted By:

- Muhammad Ali
- Muhammad Talha
- Haseeb Ahsan

Section: BSCYS-F-24-A

Table of Contents

1. Introduction
2. Motivation and Problem Statement
3. Project Objectives
4. Background Theory
5. System Architecture
6. Technology Stack
7. Project File Structure
8. Detailed Module Descriptions
9. The Five Modes
10. Implementation Details
11. Linux Commands and System Calls Used
12. How Data Flows Through the System
13. Results and Demo Outcomes
14. Security Considerations
15. Conclusion

1. Introduction

The Scheduler Manipulator is a real-time tool for manipulating and visualizing how the Linux kernel allocates CPU time to processes.

The project consists of three integrated layers: C programs at the bottom that create real CPU load and real thread contention, a Python web server in the middle that applies scheduler changes through Linux commands and system calls, and a React web dashboard on top that displays live data and provides the controls.

Unlike static simulations, every operation in this project, including priority changes, CPU affinity changes and mutex operations, is performed on a live, running Linux kernel. It was developed and tested on Debian Linux running under WSL2, which uses a genuine Linux kernel.

2. Motivation and Problem Statement

Operating systems courses often teach CPU scheduling and concepts like priority inversion through static textbook theory rather than real-world application. This project makes scheduling tangible: students can change, break and observe kernel scheduling behaviour live on screen instead of only memorizing the concepts.

3. Project Objectives

Primary objectives:

- Read live process data from the Linux kernel and display it in a web dashboard that updates in real time.
- Manipulate the kernel scheduler directly with real Linux tools: `renice`, `taskset` and `chrt`.
- Recreate a genuine priority inversion with real-time POSIX threads and a mutex, the failure mode that repeatedly reset NASA's Mars Pathfinder rover in 1997, and show how priority inheritance fixes it.
- Compare two identical processes side by side, one manipulated and one not, to prove that the manipulation is real and measurable.

Secondary objectives:

- Build a complete full-stack system that integrates C, Python and React in one application.
- Use the WebSocket protocol for real-time, two-way communication between the web frontend and the system backend.
- Provide a clear visual interface that makes kernel behaviour understandable to someone with no Linux experience.

4. Background Theory

4.1 The Linux CPU Scheduler

The Linux kernel scheduler decides how CPU time is shared when there are more runnable tasks than CPU cores. For normal tasks its default algorithm is the Completely Fair Scheduler (CFS), introduced in 2007 to replace the older $O(1)$ scheduler and share the CPU fairly according to each task's weight. (Linux 6.6 replaced CFS with the closely related EEVDF scheduler; nice values and the concepts below work the same way.)

4.2 How CFS Works

CFS tracks each task's CPU usage as a virtual runtime (`vruntime`), which grows with execution time and is scaled by the task's weight. Runnable tasks are kept in a red-black tree ordered by `vruntime`, and the scheduler runs the leftmost task (lowest `vruntime`), finding it in $O(\log n)$ time.

4.3 Nice Values

Nice values range from -20 (highest priority, largest weight) to +19 (lowest priority, smallest weight). A lower nice value makes `vruntime` grow more slowly, so the task receives more CPU time when tasks compete for the same core. Any user can lower the priority of their own process, but only root (or a process with the `CAP_SYS_NICE` capability) can set a negative nice value.

4.4 CPU Affinity

CPU affinity restricts a process to specific CPU cores, overriding the kernel's normal load balancing. Restricting affinity with `taskset` can force processes to share one busy core even while other cores are idle.

4.5 Scheduling Policies

Linux has several scheduling policies: `SCHED_OTHER` for normal CFS tasks, `SCHED_BATCH` for non-interactive jobs, and the real-time policies `SCHED_FIFO` (runs until it blocks or a higher-priority task arrives) and `SCHED_RR` (round-robin with time slices). Real-time tasks always run before normal tasks and ignore nice values. Policies are set with the `sched_setscheduler()` system call, which the `chrt` command and the pthread scheduling attributes use.

4.6 Processes and Threads

A process is a running program with its own memory space; threads are execution units inside a process that share its memory. The Linux kernel schedules individual threads, so the threads of one program can have different priorities and compete with each other for the CPU.

4.7 Mutexes

A mutex (mutual exclusion lock) allows only one thread at a time into a critical section. A thread that tries to lock a mutex that is already held blocks and stops using the CPU until the lock is released.

4.8 Priority Inversion

Priority inversion happens when a low-priority task holds a mutex that a high-priority task needs, and a medium-priority task (which does not need the mutex) preempts the low-priority task. The

low-priority task cannot finish and release the lock, so the high-priority task is effectively blocked by a lower-priority one for as long as the medium task keeps running.

4.9 Priority Inheritance

Priority inheritance fixes this by temporarily raising the lock holder to the priority of the highest-priority task waiting for the lock. The medium task can then no longer preempt it, so it finishes its critical section and releases the mutex quickly. POSIX enables this per mutex with the `PTHREAD_PRIO_INHERIT` protocol.

4.10 WebSocket Protocol

The WebSocket protocol keeps one persistent, full-duplex TCP connection open for real-time two-way data exchange without the overhead of repeated HTTP requests. In this project the dashboard sends control commands over it, and asks for fresh process statistics every 500 ms.

5. System Architecture

Layer 1: React frontend (Vite dev server, port 5173). The browser dashboard keeps a WebSocket connection to the Python backend and renders the live process table, the throughput chart, the inversion log and the event log. User actions are sent to the backend as JSON commands.

Layer 2: Python backend (port 5000). A Flask server with the flask-sock WebSocket extension translates JSON commands into scheduler actions, starts and stops the C programs, and samples live process statistics with the `psutil` library. It listens on `127.0.0.1` only, because it can change the priority of any process.

Layer 3: C programs. `worker` runs an endless CPU-bound loop and reports its throughput. `mutex_demo` starts three real-time threads with different priorities on one CPU core to recreate priority inversion, optionally with priority inheritance.

Layer 4: Linux kernel. The scheduler is the target of every operation. It receives the changes through real system calls (`setpriority`, `sched_setaffinity`, `sched_setscheduler`) and changes how it allocates CPU time.

Communication flow: the dashboard sends a JSON command over the WebSocket; the backend extracts the action and PID, runs the matching Linux command, and reports success or failure back. Every 500 ms the dashboard requests statistics; the backend reads them from `/proc` through `psutil` and sends them back as JSON to refresh the table and chart.

6. Technology Stack

- **React and Vite:** JavaScript library and build tool used for the dashboard, giving real-time UI updates without page reloads.
- **Recharts:** React charting library used for the live line chart that compares the throughput of two workers.
- **Python, Flask and flask-sock:** lightweight web framework and WebSocket extension for the backend server.

- **psutil:** Python library that reads the `/proc` filesystem to get process data such as PID, nice value, CPU usage, status and CPU affinity.
- **C with POSIX threads:** used for the workloads, for direct access to Linux system calls and to real-time threads and mutexes.
- **WebSocket:** low-latency two-way protocol between dashboard and backend.

7. Project File Structure

scheduler-manipulator/		
├── c/		
│ ├── worker.c	CPU-bound workload that reports its throughput	
│ ├── mutex_demo.c	real priority inversion (+ <code>--inherit</code> fix)	
│ └── Makefile	builds both programs	
├── backend/		
│ ├── app.py	Flask WebSocket server	
│ ├── scheduler.py	SchedulerManipulator class (<code>renice/taskset/chrt</code>)	
│ └── requirements.txt	Python dependencies	
├── frontend/		
│ ├── src/App.jsx	the whole React dashboard	
│ ├── src/main.jsx	React entry point	
│ ├── src/index.css	global styles	
│ ├── index.html		
│ ├── package.json	Node dependencies and scripts	
│ ├── vite.config.js	Vite build configuration	
└── README.md		

The compiled programs (`c/worker`, `c/mutex_demo`) and `frontend/node_modules` are not stored in the repository; they are created with `make` and `npm install`.

8. Detailed Module Descriptions

8.1 worker.c

Repeats a fixed block of work (5 million floating-point multiplications) forever. After each block it measures the elapsed time with `clock_gettime(CLOCK_MONOTONIC)` and prints its throughput in blocks per second. The backend reads this from the program's standard output. When the scheduler gives the worker less CPU time, the number drops.

8.2 mutex_demo.c

Recreates priority inversion with real kernel scheduling. It pins itself to CPU core 0 with `sched_setaffinity()` so that all its threads compete for one CPU, then starts three `SCHED_FIFO` threads: LOW (priority 10), MED (priority 20) and HIGH (priority 30). LOW locks the shared mutex and needs 1000 ms of CPU time; HIGH then needs the same mutex; MED, which never uses the mutex, needs 2000 ms of CPU time. Work is measured in each thread's own CPU time (`CLOCK_THREAD_CPUTIME_ID`), so time spent preempted does not count as work done.

Every event is printed with a timestamp, and a final `RESULT` line reports how long HIGH waited for the mutex. Run with `--inherit`, the mutex is created with `PTHREAD_PRIO_INHERIT`. Because it uses real-time scheduling, the program needs root and prints a clear error otherwise.

8.3 scheduler.py

Defines the SchedulerManipulator class. Its methods run `renice`, `taskset` and `chrt` through `subprocess.run` with argument lists (no shell), and return whether the command succeeded together with its output. If the backend is not running as root and a command fails, it is retried once through `sudo -n`, which works when `sudoers` allows these commands without a password. It also uses `psutil` to collect live process statistics.

8.4 app.py

The Flask application. It serves the WebSocket endpoint `/ws`, routes each JSON action to its handler, starts the C programs with `subprocess.Popen` (paths are relative to the project folder), and runs background threads that read worker throughput and stream the inversion demo's output. Successes and failures are both reported to the dashboard. When the dashboard disconnects, all workers it started are stopped.

8.5 App.jsx

The single-file React dashboard. It opens the WebSocket connection, requests statistics every 500 ms, and switches its controls between the five modes. It shows the live process table (PID, name, nice value, CPU %, allowed cores, status and throughput), the Compare chart, the colour-coded inversion log and an event log in which failed actions appear in red.

9. The Five Modes

9.1 Observe Mode

Shows a live process table refreshed every 500 ms. With no workers running it lists the 20 processes using the most CPU; once workers are spawned it shows those workers, including each worker's own throughput.

9.2 Inflate Mode

Changes a worker's nice value to -15 with `renice`, so CFS gives it a larger share of the CPU. The new nice value appears immediately in the table. The gain in throughput is visible when the worker competes with another process for the same core, which is what Starve and Compare modes set up.

9.3 Starve Mode

Uses `taskset` to bind workers to CPU core 0. When both workers are pinned there, they have to share one core even though the other cores are idle, and each worker's throughput drops to about half. Restore returns a worker to nice 0, the normal policy and all cores.

9.4 Inversion Mode

Runs `mutex_demo` and streams its colour-coded log to the dashboard. **Run inversion** shows HIGH stuck behind MED; **Run with priority inheritance** runs the same scenario with `--inherit` and shows HIGH getting the mutex much sooner.

9.5 Compare Mode

Starts two identical workers, "normal" and "boosted", and pins both to core 0 so they compete for the same CPU. Their throughput is plotted live; the lines start level and split apart once "boosted" is set to nice -19.

10. Implementation Details

10.1 WebSocket Implementation

On load, the React component connects to `ws://localhost:5000/ws` and starts a `setInterval` timer that sends `get_stats` every 500 ms. The flask-sock handler loops on incoming messages, decodes each JSON action and calls the matching handler. Because several threads send on the same connection, sends are protected by a lock.

10.2 Process Spawning

The backend starts the worker binary with `subprocess.Popen`, keeps the process handle in a dictionary, and starts a daemon thread that reads the worker's output and stores its latest throughput. Compare mode passes a core number when spawning, so the new worker is pinned with `taskset` straight away.

10.3 Output Streaming for the Inversion Demo

When the demo is started, a background thread launches `mutex_demo` (through `sudo -n` when the backend is not root) and sends each output line to the dashboard as a separate WebSocket message, followed by the program's exit code. The log therefore appears live, line by line.

10.4 Priority Manipulation

Priority and affinity changes run `renice -n <value> -p <pid>`, `taskset -cp <core> <pid>` and `chrt -o -p 0 <pid>`. Negative nice values and real-time scheduling need root, so the backend is either started with `sudo`, or the Linux sudoers file is configured so that these specific commands can run without a password:

```
<user> ALL=(root) NOPASSWD: /usr/bin/renice, /usr/bin/taskset, /usr/bin/chrt, \
/path/to/scheduler-manipulator/c/mutex_demo
```

If neither is set up, the action fails and the dashboard shows the error instead of reporting success.

10.5 Live Chart Data

The dashboard keeps the 30 most recent data points, each with a timestamp and every worker's throughput. New data is appended and the oldest point dropped, giving a rolling-window chart in Recharts.

11. Linux Commands and System Calls Used

- **renice:** changes a running process's nice value through the `setpriority()` system call (Inflate, Compare, Restore).

- **taskset:** changes a process's CPU affinity through `sched_setaffinity()` (Starve, Compare, Restore).
- **chrt:** sets a process's scheduling policy through `sched_setscheduler()`; Restore uses it to put a process back on the normal `SCHED_OTHER` policy.
- **pthread_attr_setschedpolicy / pthread_setschedparam:** give the demo's threads the real-time `SCHED_FIFO` policy and priorities, also through `sched_setscheduler()`.
- **sched_setaffinity():** called directly by `mutex_demo` to pin itself to core 0.
- **pthread_mutex_lock / unlock and pthread_mutexattr_setprotocol:** thread synchronisation built on the `futex()` system call; the protocol attribute enables `PTHREAD_PRIO_INHERIT`.
- **clock_gettime:** `CLOCK_MONOTONIC` timestamps for throughput and the demo log, and `CLOCK_THREAD_CPUTIME_ID` to measure each thread's own CPU time.
- **/proc filesystem:** read by `psutil` to obtain live process information (`/proc/[pid]/stat` and related files).

12. How Data Flows Through the System

User clicks Inflate workerA. The React click handler sends `{"action": "inflate", "pid": <pid>, "nice": -15}` over the WebSocket.

Python receives and processes it. The handler parses the JSON, identifies the inflate action and calls `manip.inflate_priority(pid, -15)`, which runs `renice -n -15 -p <pid>`. The kernel performs the `setpriority()` system call and updates the task's weight. The backend sends back an ok event (or an error event with the reason), which appears in the event log.

500 ms statistics cycle. The dashboard's timer sends `get_stats`; the backend calls `manip.get_process_stats()`, `psutil` reads the new nice value (-15) and other data from `/proc`, and the JSON reply re-renders the process table.

13. Results and Demo Outcomes

The results below were measured with the final code on a 2-core Linux machine. Throughput is in blocks per second; exact numbers depend on the CPU, but the pattern is the same.

Mode	What was done	Measured result
Observe	Live process table	Updates every 500 ms with PID, nice, CPU %, cores, status and worker throughput
Inflate	Worker reniced from 0 to -15	Nice column changed to -15 (kernel priority 20 to 5); confirmed in the event log
Starve	Two workers pinned to core 0	About 65 blocks/s each when free; about 36 each when sharing core 0

Mode	What was done	Measured result
Starve + Inflate	One of the pinned workers reniced to -15	68 blocks/s for the inflated worker vs 2.5 for the other
Compare	"boosted" reniced to -19, both on core 0	Before: 36 vs 35. After: 67 (boosted) vs 1.1 (normal)
Inversion	mutex_demo without inheritance	HIGH waited about 2950 ms: blocked while MED (lower priority) ran
Inversion fix	mutex_demo --inherit	HIGH waited about 800 ms: only LOW's remaining critical section

Without inheritance, the log shows MED running from 300 ms to about 2400 ms while LOW, which holds the mutex, cannot run; HIGH only gets the mutex at about 3150 ms. With inheritance, LOW is boosted to HIGH's priority as soon as HIGH blocks, finishes at about 1000 ms, HIGH gets the mutex immediately, and MED runs afterwards.

14. Security Considerations

- The backend can change the priority of any process, so it listens on 127.0.0.1 only and is not reachable from the network.
- Commands are run with argument lists through `subprocess.run`, never through a shell, and PIDs, nice values and core numbers are converted to integers first, which prevents command injection.
- When passwordless sudo is used, it is limited to the exact commands the tool needs (`renice`, `taskset`, `chrt` and `mutex_demo`), not full root access.

15. Conclusion

The Scheduler Manipulator turns CPU scheduling from abstract textbook theory into a concrete, interactive system by manipulating a live Linux kernel through real system calls. Its main achievement is a genuine recreation of the Mars Pathfinder priority inversion with real-time threads, together with the priority inheritance fix, with the difference measured in milliseconds. The project also shows a complete full-stack design that connects low-level C system programming with Python and React through WebSockets. All five modes work and produce measurable effects, meeting the objective of making core kernel behaviour visible and measurable on demand.